

High Performance Programming

Lecture 5

“Practical aspects of high-performance programming”

Lecturer: Stefan Nordborg Eriksen



AALBORG
UNIVERSITET

Contents

1. Hardware platforms: CPU and GPU

2. Software

3. Vectorization

4. Vectorization in Python

5. Applications and examples

6. Summary and exercises

1. Hardware

Hardware platforms

Computer architecture:

- Data types
- Memory and registers
- Memory consistency and addressing modes
- Input/output model
- Instruction set

Traditional classification based on the set of instructions

Reduced Instruction Set Computing (RISC):

Highly optimized, only some specific instructions to access memory

Complex instruction Set Computing (CISC):

1 instruction $\rightarrow$ X operations

Hardware platforms (cont.)

CISC (x86)

PC: Intel and AMD CPUs

Graphics accelerators (GPUs)

How to choose?

Clock cycle = $1 / \text{frequency}$

More cores = better multitasking

More memory = faster access to data

32-bit and 64-bit systems

Trend: Lower frequency but more cores

RISC

Smartphones, tablets, embedded systems, and supercomputers(?)

ARM and AVR

CPLDs and FPGAs

- VHDL or Verilog
- Gates, flip-flops, multiplexers, arithmetic units

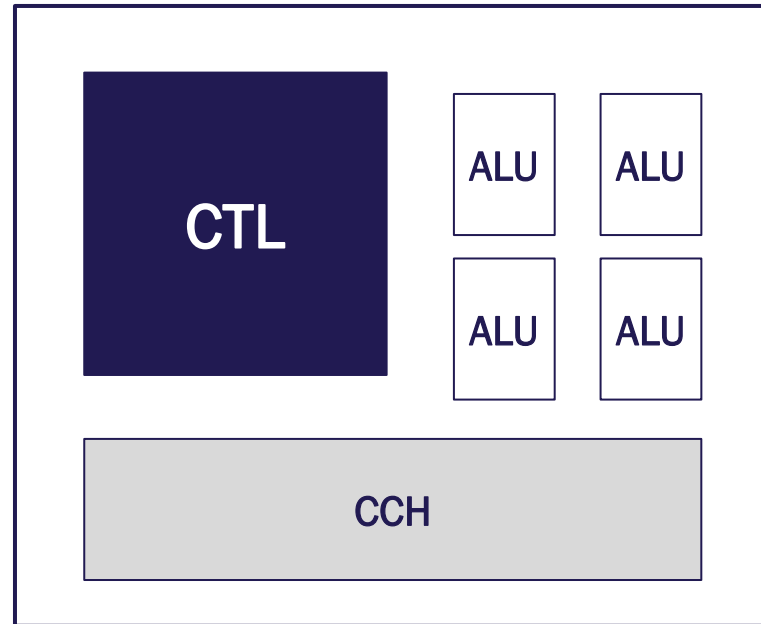
Great for real-time systems

Few things happening besides your code

Precise execution time per instruction

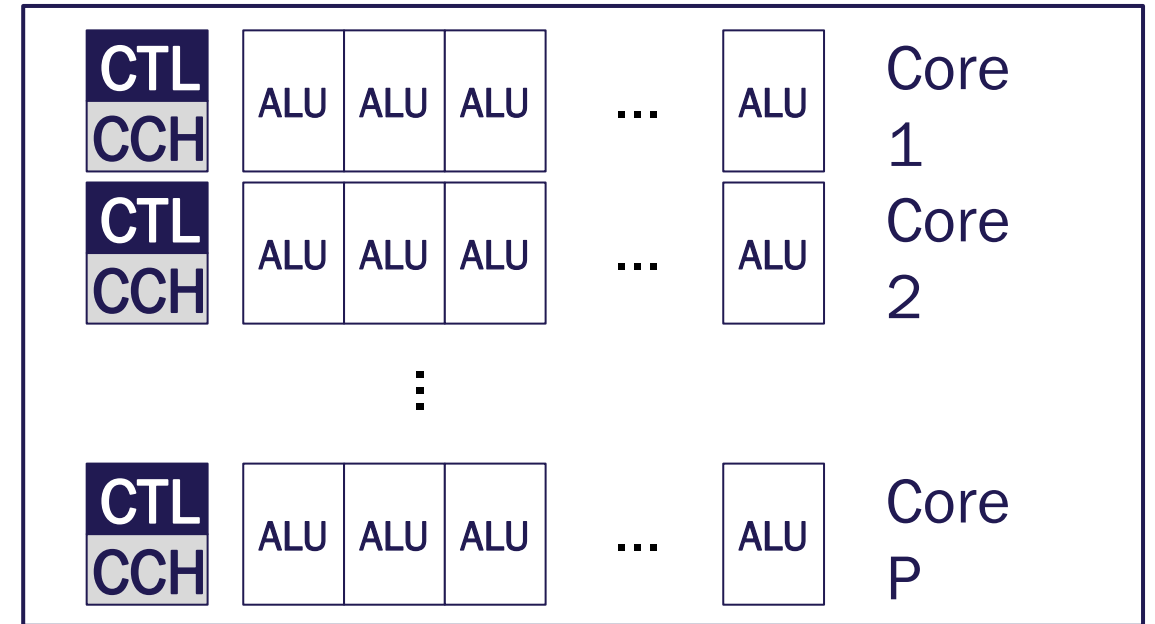
Architectures: CPU vs. GPU

CPU



- Large area to control (CTL) and cache (CCH)
- Few cores
- Few logic units (ALUs)

GPU



- Low control capabilities
- Large number of cores
- Massive number of ALUs

Purpose of the GPU

Perform a large number of similar operations at once

Same Instruction Multiple Data (SIMD)

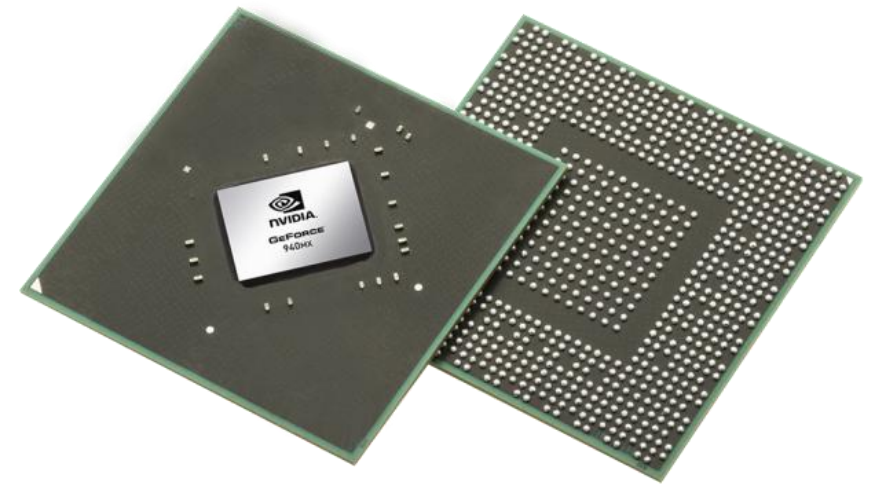
Elements in each core work synchronously but distinct operations can be done in each core

Great for problems with a large number of common operations

Video and image processing, simulations, and machine (deep) learning

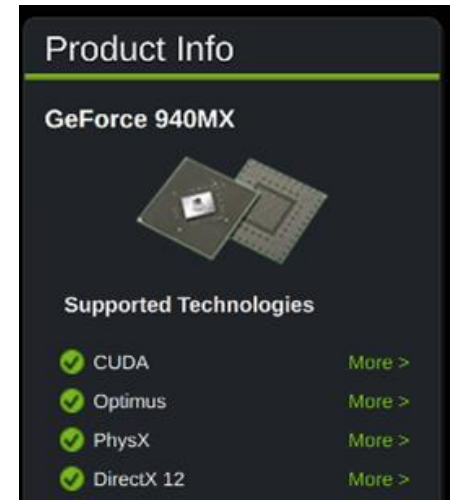
Not good if

- Large amounts of control operations are needed
- Communication from CPU to GPU becomes the bottleneck
- Complex memory access patterns are needed



CNT job server vs. Ramoni's laptop

	CNT job server	My laptop
Operating system	Ubuntu 16.04.4 (64-bit)	Ubuntu 20.04.2 LTS (64-bit)
Processor	Intel Xeon(R) Gold 6132 x52	Intel® Core™ i7-7820HQ
Clock frequency	2.60 GHz	2.90 GHz
Total cores	14	4
Max. threads	28	8
RAM	251.6 GB	15.2 GB
GPU	NVIDIA Quadro GV100 5120 CUDA cores	NVIDIA GeForce 940MX 384 CUDA cores



Top 500 list



Ranking of the 500 fastest supercomputers: Latest release is November 2024

Go into the list [here](#) and find the following data. Try also to find Gefion!

1.

2.

?. Gefion

Operating system

No. processor cores

Clock frequency

RAM

GPU

MPI

Compute Unified Device Architecture (CUDA)

Parallel computing platform and programming model

Supported languages: C, C++, Fortran, Python and MATLAB

CUDA Toolkit (not a Python package)

Libraries, compiler, development tools

Some CUDA libraries include:

cuBLAS: Basic linear algebra (BLAS) library

cuFFT: Library for Fast Fourier Transforms

CUDA Math Library: Standard mathematical function library

nvJPEG: For JPEG decoding

NVIDIA Video Codec SDK: Hardware-accelerated video encode and decode

Memory access trade-offs

Not all memory is the same

Disk memory: TBs of capacity

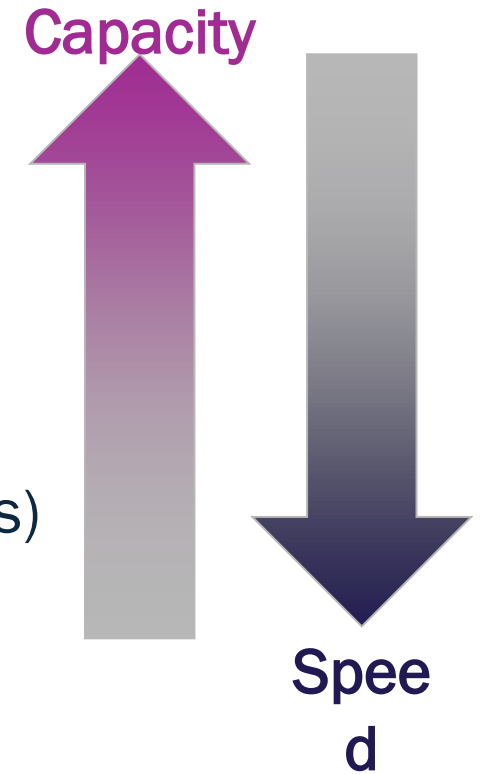
Main memory: GBs of capacity

Volatile: Random Access Memory (RAM)

Non-volatile: Flash and Read-Only Memory (ROM)

Cache: Few MB of capacity. Several levels

Registers: Inside the CPU used for temporary storage (local variables)



2. Software

Software: The programming language

Compiled languages: C, C++

The fastest of the fast

Compile, distribute, execute

Compilers:

Microsoft Visual Studio, gcc /g++

Implemented with interpretation: Python, JavaScript, PHP, Perl

Based on intermediate code: C#, Visual Basic

Just-in-time compilation: Numba (Python package)

Stack

Variables and objects declared inside functions

First-in-last-out

Allocation of memory per function call; memory is freed upon termination

Most efficient because same addresses are reused

Analogy to recursion: Fibonacci sequence

```
int fib(int n){  
    if (n <= 1)  
        return n;  
    return fib(n-1) + fib(n-2);  
}
```

Global and static variables (C/C++)

Global variables

- Declared outside of any function
- Can be accessed from anywhere
 - Potentially problematic
- Both allocated in a static part of the memory, the data segment
- Values persist across calls

Good for:

- Constants that never change
- Initialized variables that may be changed
- Non-initialized variables that may be changed

Caution: both global & static variables persist and use space throughout the program's execution

Static variables

- Declared inside function or file scope
- Access limited to function block

Variables inside a class

Object-oriented programming: attributes

Stored in the order in which they appear

A class member variable with the **static** modifier will be stored in static memory and has only one instance, shared across class instances

This is a good alternative to global variables

Other non-static members will be stored for each instance of the class

Biggest time consumers and big $\mathcal{O}$ notation

Different operations take different number of clock cycles to execute

Distinct times to fetch the data add implementation-specific aspects to execution time

Operation	Clock cycles	
	Integer	Float
Addition, subtraction, comparison	1	3 - 6
Multiplication	3 - 4	4 - 8
Division	40 - 80	14 - 45

Big $\mathcal{O}$ notation illustrates the asymptotic behavior, when the problem size becomes large

Python basics

“All data in a Python program is represented by objects or by relations between objects.”

—<https://docs.python.org/3/reference/datamodel.html#>

Python is an **interpreted** language: code is translated and executed without compilation

It has several modules/packages to make it fast:

- Numpy
- Numba
- Pytorch
- PyOpenCL



3. Vectorization

Maximum speedup according to Amdahl's law

Parallel programming has its limits

A speed(up) limit that we cannot exceed

Maximum speedup is p , the no. of processors

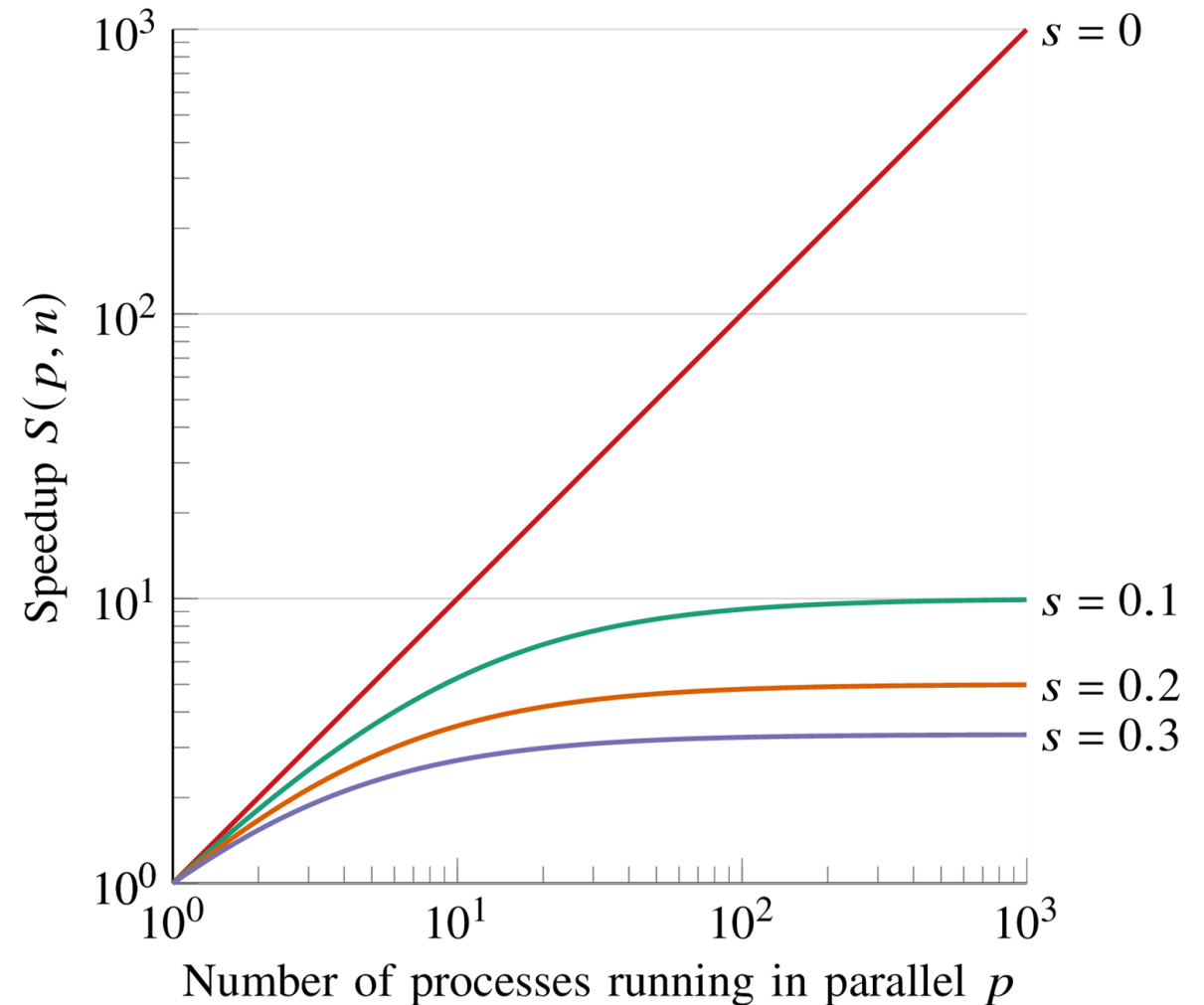
This is assuming

- 1.No communication overhead
- 2.A problem of fixed size n
- 3.Full parallelism
- 4.Best serial implementation

A fraction s of the code cannot be parallelized

s depends on the problem size

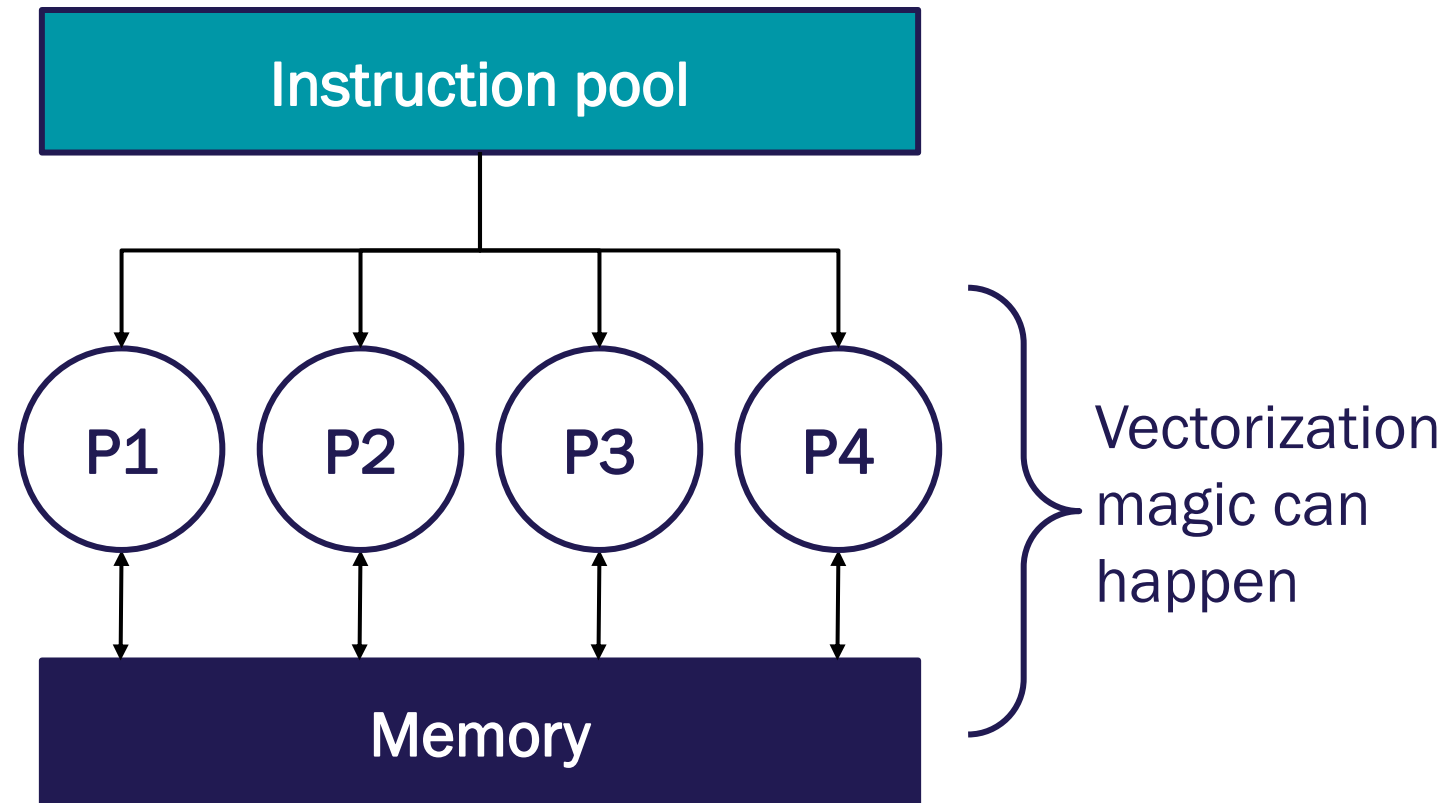
Communication overhead increases with p



Vectorization

Turn multiple independent instructions into single instruction multiple data (SIMD) form

Use greatly optimized vector operations at the compiler/interpreter



Vectorization

Turn multiple independent instructions into single instruction multiple data (SIMD) form

Use greatly optimized vector operations at the compiler/interpreter

Example: Hadamard (element-wise) product $\mathbf{c} = \mathbf{a} \circ \mathbf{b}$ in a loop. **Complexity?**

```
for i in range(N) :
```

```
    c[i] = a[i]*b[i]
```

$$c_i = a_i \times b_i$$

In the loop, $a[0]*b[0]$ is calculated before $a[1]*b[1]$ but this is **not required**

We can **unroll the loop**; break the vectors $\mathbf{a}$ and $\mathbf{b}$, which have length N , into q segments V of length M

$$V_i \leftarrow [a_{iM}, a_{iM+1}, \dots, a_{iM+M-1}]$$

$$N = qM + r$$

Loop unrolling

Technique that aims to reduce execution time at the expense of increasing the size

Instead of going through all the indices of a loop, we make small batches

Example:

Original loop

```
for i in range(N):  
    c[i] = a[i]*b[i]
```

Advantages:

You can save time by avoiding increasing the counters

Easy parallelization

Unrolled loop

```
for i in range(0,N,5):  
    c[i] = a[i]*b[i]  
    c[i+1] = a[i+1]*b[i+1]  
    c[i+2] = a[i+2]*b[i+2]  
    c[i+3] = a[i+3]*b[i+3]  
    c[i+4] = a[i+4]*b[i+4]
```

Loop unrolling

The loops cannot be unrolled if there are data dependencies

In the previous example, there are no data dependencies

What if we need the dot product of the vectors? **Complexity?**

```
for i in range(N) :
```

```
    c += a[i]*b[i]
```

$$c = \sum_{i=0}^{N-1} a_i \times b_i$$

We can perform the Hadamard product $c[i] = a[i] * b[i]$ and then sum

We look for ways to perform multiple operations simultaneously, using efficient algorithms

We'll come back to this example...

Branching

Dealing with if statements is tricky

```
for i in range(N) :
```

```
    if e(a[i]) :
```

```
        c[i] = f(a[i])
```

```
    else:
```

```
        c[i] = g(a[i])
```

What should we do here?

Automatic and manual vectorization

Automatic: Compilers and interpreters come with automatic vectorization capabilities

Attempt to optimize performance or code size based on the knowledge of the program.

A terrible lie: “No need to know the problem structure, the software is smart”

Manual (not a standard term): The programmer decides which operations to vectorize

Use vectorized functions for the specific operations

Requires decomposing the problem to single-instruction multiple data (SIMD)

Automatic vectorization in C: GCC/G++

Optimization flags: the compiler attempts to improve the performance and/or code size

Cost: compilation time and, possibly the ability to debug the program.

Compiling multiple files at once:

Allows the compiler to use information gained from all of the files.

<https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html>

Automatic vectorization in C: GCC/G++

Summary of optimization flags for GCC/G++ `g++ -flag code.cpp -o exec`

Summary of options for `-flag` in the form `-OX (Oh-X)`

X = 0: No optimization.

X = 1: Try to reduce code size and execution time, without hurting compilation time.

X = 2: Performs nearly all optimizations that do not involve a space-speed tradeoff.

X = 3: Optimize even more. Automatic vectorization; also with `-ftree-vectorize`.

More details at <https://gcc.gnu.org/projects/tree-ssa/vectorization.html#using>

More on optimization options <https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html>

4. Vectorization in Python

Numpy universal functions (ufuncs)

Functions that operate in arrays in an element-by-element fashion.

Support array broadcasting, type casting, and several other standard features.

That is, a ufunc is a “vectorized” wrapper for a function that:

- 1.Takes a fixed number of specific inputs
- 2.Produces a fixed number of specific outputs.

There are more than 60 ufuncs defined in Numpy

Many of the built-in functions are implemented in compiled C code

Usage: `numpy.ufunc()`

Generalized universal functions (gufuncs) can operate in more than one dimension



Numpy universal functions (ufuncs)

All ufuncs have four methods

Reduce: To perform the operation along one axis and reduce the dimension

Accumulate: To accumulate the result of applying the operator to all elements.

Reduceat: To perform the operation in slices over a single axis

At: To perform the operation at specific indices of an array

Numba just-in-time (JIT) compilation

“Just-in-time [JIT] compiler for Python that works best on code that uses NumPy arrays and functions, and loops.”

[— A ~5 minute guide to Numba](#)

Decorator design pattern

Add functionality to an object without having to make a new one

```
@jit(nopython=True)           # Compile the decorated
function                       function
def function(input1, input2, ...)
```

The function compiles the first time is called
(time cost first time, but fast in subsequent calls)



Numba vectorize with ufuncs

Another decorator

```
@vectorize(signature, target='target')  
def function(input1, input2, ...):
```

The function is **compiled into a Numpy ufunc** the first time is called.

Then, it will be executed as defined in target

Signature: `['output_type(input1_type, input1_type, ...)']`

Example: `['float32(float32, float32)']`

Numba vectorize example

```
from numba import vectorize, float64

@vectorize([float64(float64, float64)])

def f(x, y):
    return x + y
```

Note that you need to specify the input and output data types

You can define several signatures to cover different possible input types

```
@vectorize([int32(int32, int32),
             int64(int64, int64),
             float32(float32, float32),
             float64(float64, float64)])
```

Numba vectorize: target

What is target?

The place to execute the operations:

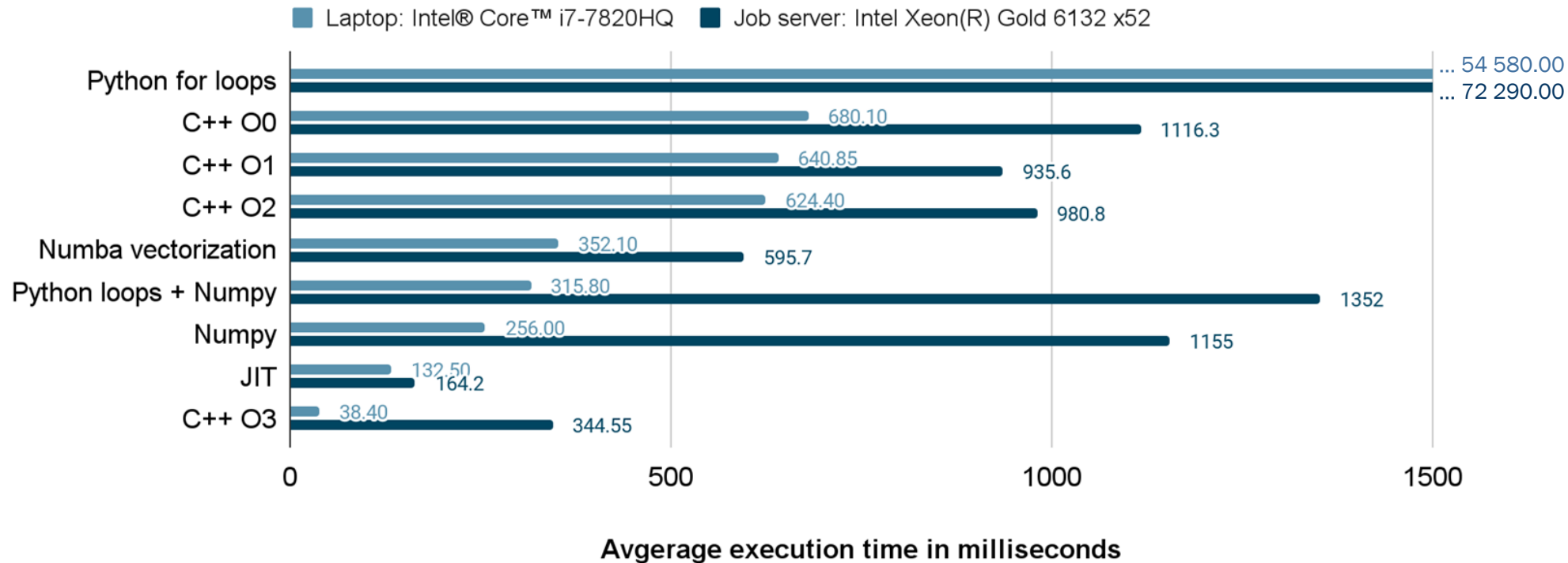
cpu: One core of the CPU

parallel: Multiple cores of the CPU

cuda: GPU (NVIDIA only)

How powerful is vectorization?

Multiply a vector of length $N = 10\,000$ by an $N \times N$ matrix of integers



Numba vectorize with gufuncs

Decorator @guvectorize

Signature is different here: [(input1_type, input1_type, ..., output_type)]

We need to define the layout: '(n_elements_input_1), (n_elements_input_2), ... -> (n_elements_output)'

Example:

```
@guvectorize([(int64[:], int64, int64[:])], '(n), ()->(n)')
```

```
def g(x, y, res):
```

```
    for i in range(x.shape[0]):
```

```
        res[i] = x[i] + y
```

5. Applications and examples

Matrix-vector multiplication and the DCT

The discrete cosine transform (DCT) is widely used to perform spectral analysis

- Images
- Video
- Audio

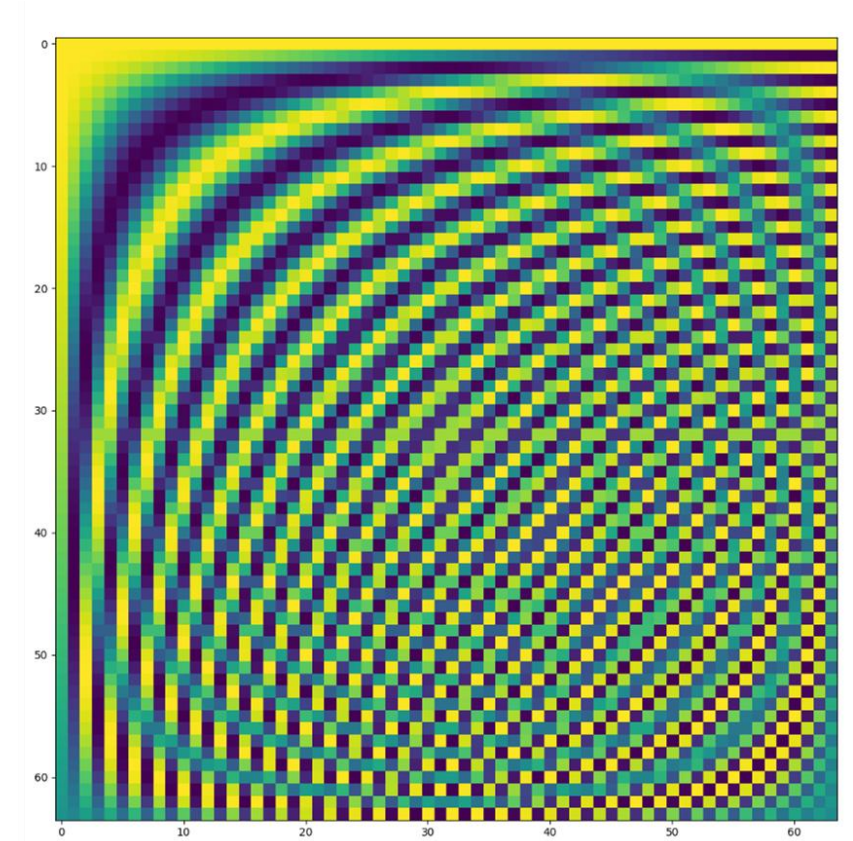
Compression

Real-valued version of the Fourier transform

The standard DCT is defined as $y_k = 2 \sum_{n=0}^{N-1} x_n \cos\left(\frac{\pi k(2n+1)}{2N}\right)$

What is the complexity of this in big $\mathcal{O}$ notation?

There is a **Fast Cosine Transform**



Example on value approximation

Approximate the value of π

Method:

$$A_c(r) = \pi r^2$$

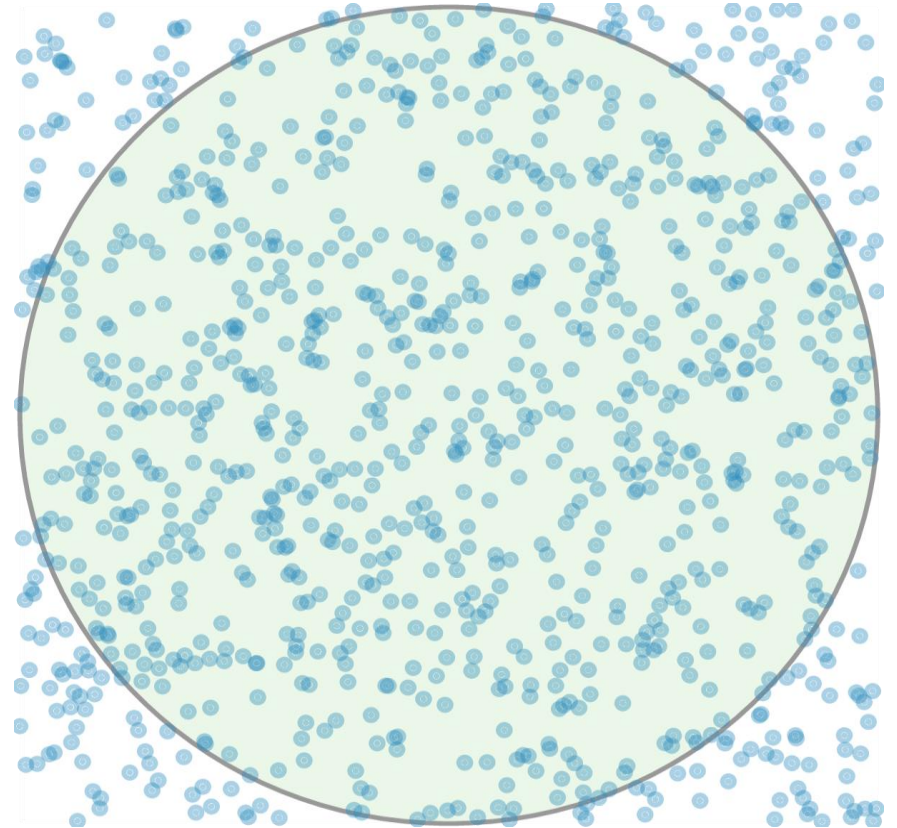
$$A_s(d) = d^2$$

$$\frac{A_c(r)}{A_s(d)} = \frac{\pi r^2}{4r^2} = \frac{\pi}{4}$$

Generate random points inside a square area

The probability they fall in the circle is $\frac{\pi}{4}$

$$\rightarrow \pi \approx \frac{P_{circle}}{P_{square}}$$



Example on value approximation

Approximate the value of π

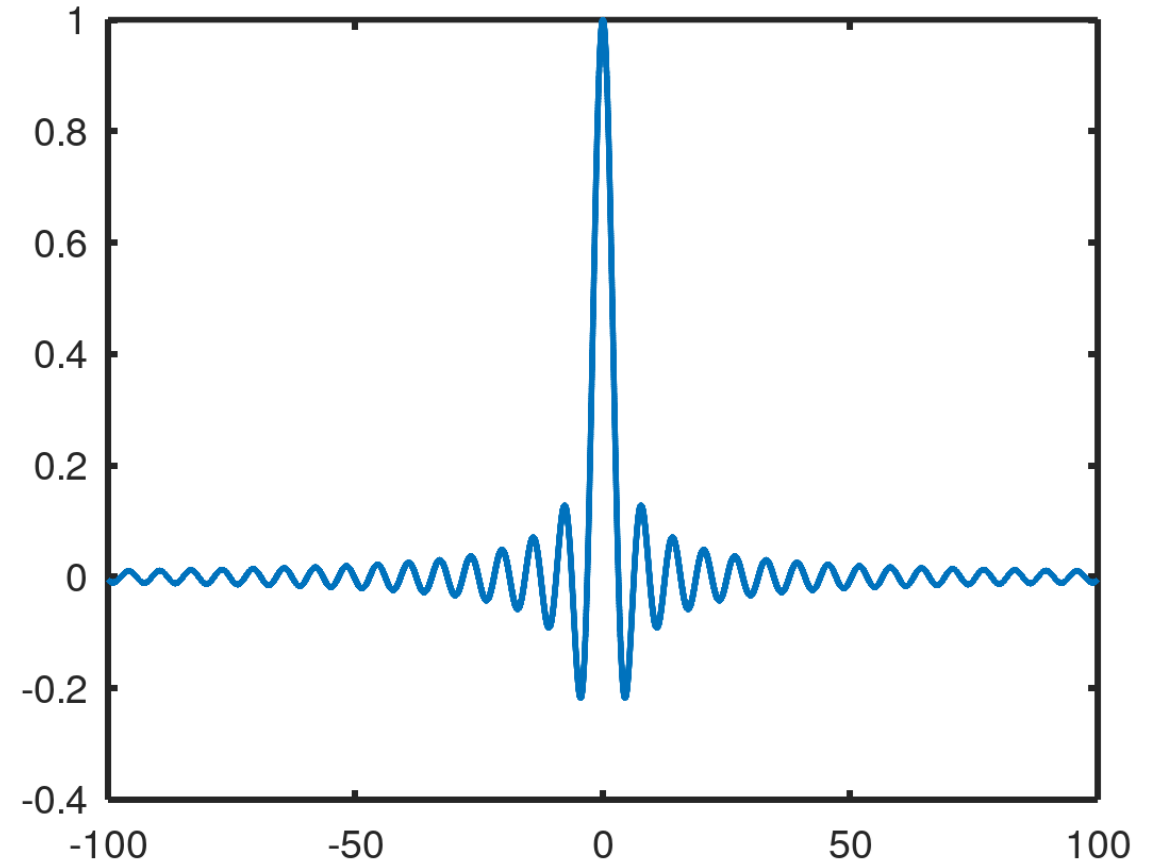
Method: $\int_{-\infty}^{\infty} \text{sinc}(x) = \pi$

$$\text{sinc}(x) = \begin{cases} 1, & \text{if } x = 0, \\ \frac{\sin(x)}{x}, & \text{otherwise} \end{cases}$$

$$\int_a^b f(x) dx = \lim_{\|\Delta x\| \rightarrow 0} \sum_{i=0}^{\lfloor (b-a)/\Delta x \rfloor} f(a + i\Delta x) \Delta x$$

1. Define Δx

2. Calculate and add the values for each point



Example on value approximation

Approximate the value of π

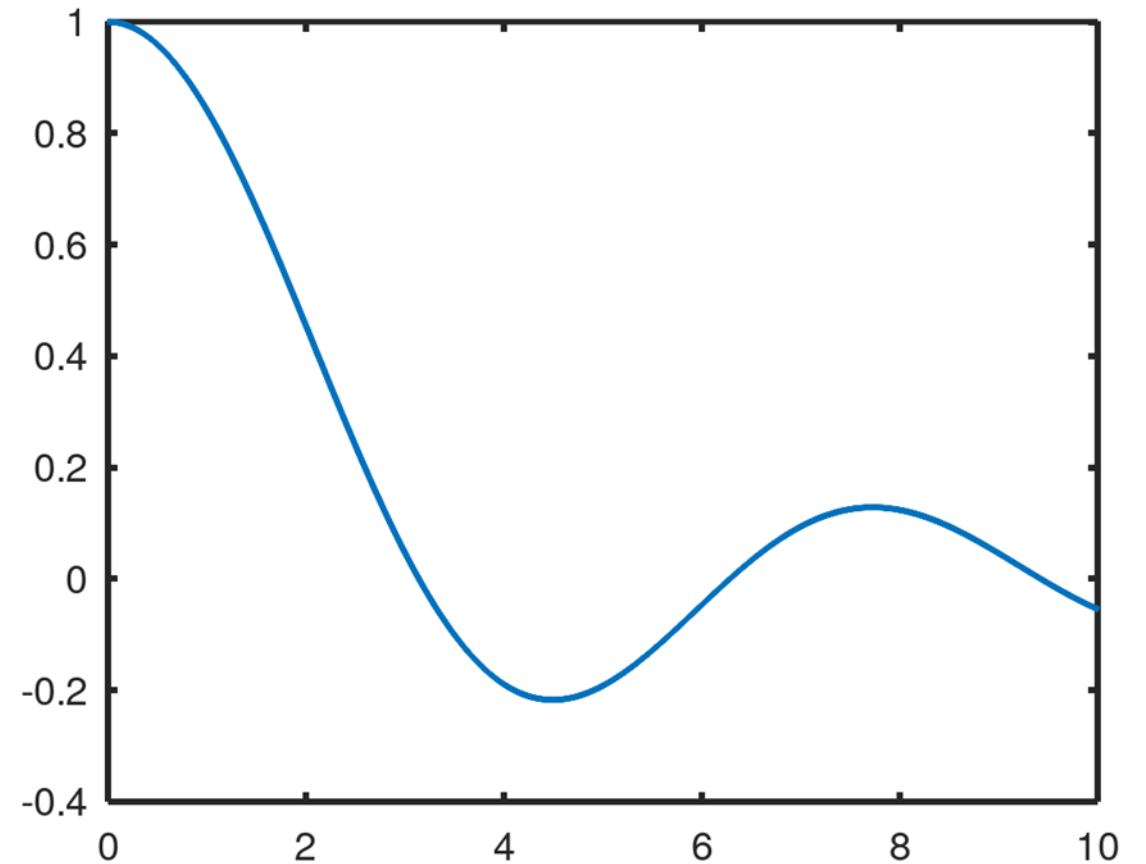
Method: $\int_{-\infty}^{\infty} \text{sinc}(x) = \pi$

$$\text{sinc}(x) = \begin{cases} 1, & \text{if } x = 0, \\ \frac{\sin(x)}{x}, & \text{otherwise} \end{cases}$$

$$\int_a^b f(x) dx = \lim_{\|\Delta x\| \rightarrow 0} \sum_{i=0}^{\lfloor (b-a)/\Delta x \rfloor} f(a + i\Delta x) \Delta x$$

1. Define Δx

2. Calculate and add the values for each point



6. Summary and exercises

Summary

Python is considered to be a slow programming language

But it has several packages to speed up the code:

- Just-in-time (JIT) compilation
- Numpy: greatly efficient vectorized operations written in C
- Manual vectorization with Numba

The execution time can get close to the most optimized version compiled in C

Avoid for loops: these are always candidates for optimization

Exercises

1. The code I gave you for vector-matrix multiplication is **wrong**: results don't match. Correct it.
2. Extend the code to multiplying random float numbers with normal distribution. Use `np.random.randn(size)`. Do you encounter problems?
1. Use gufuncs with `@vectorize` to calculate the vector matrix product $\mathbf{y} = \mathbf{x}\mathbf{A}$
 - a. Between using ufuncs and gufuncs, which one gives a shorter execution time?
2. Go to the function that calculates the product with Numba and JIT
 - a. Change `parallel` to `True`. How does the execution time increase?
 - b. Exchange the order of the two for loops. What is the impact of performance and why?
3. Use JIT, Numpy, and Numba (manual and automatic) vectorization to calculate the value of π . Which of the two methods gives better results?

That's it for today